Del modelo conceptual al modelo físico

Orden de SELECT, modelos de datos y Forward Engineering y Reverse Engineering

Semana 10 - Sesión 20

Rel Guzman



Logro de la sesión

Al finalizar, podrás explicar el **orden lógico de un SELECT**, distinguir los modelos **conceptual, lógico y físico**, y reconocer cómo **Forward Engineering** y **Reverse Engineering** relacionan el diseño con la base de datos. También compararás el modelo físico en **SQL Server, MySQL y SQLite**.

Revisión de la sesión anterior: orden de SELECT

```sql
SELECT c.nombre, COUNT(*) AS pedidos
FROM cliente c
JOIN pedido p ON c.id = p.cliente_id
WHERE p.fecha >= '2026-01-01'
GROUP BY c.nombre
HAVING COUNT(*) >= 2
ORDER BY pedidos DESC
LIMIT 5;
```

- qué columnas o cálculos se **muestran**
- la tabla **principal**
- qué otra tabla se **une**, y por dónde
- qué **filas** entran
- cómo se **agrupan**
- qué **grupos** se quedan
- en qué **orden** salen
- cuántas filas, **como máximo**

- `SELECT` indica qué mostrar. `FROM` se usa al consultar tablas. Lo demás se agrega si hace falta.
- Si se usan, se escriben **siempre en este orden**.
- `SELECT DISTINCT` quita las filas **repetidas** del resultado.

Se lee: «los 5 clientes con más pedidos este año, entre los que tienen 2 o más».

Se escribe en un orden, se resuelve en otro

Este es el **orden lógico** de la consulta. El motor puede usar otro plan físico para ejecutarla:



- Por eso `WHERE COUNT(*) >= 2` da error: en el paso 2 los grupos **todavía no existen**.
- Y por eso `ORDER BY pedidos` funciona: en el paso 6 el `SELECT` **ya puso** ese nombre.

Ojo con esto

Si aparece `DISTINCT`, quita duplicados después de `SELECT` y antes de `ORDER BY`. Aquí usamos `LIMIT`, como en MySQL y SQLite.

¿Por qué da error `WHERE total > 1`?

Queremos mostrar los desarrolladores que tienen **más de un videojuego**.

```sql
SELECT developer, COUNT(*) AS total
FROM videogames
WHERE total > 1
GROUP BY developer;
```

- `AS total` pone un **nombre al resultado del conteo**. No crea una columna en `videogames`.
- `WHERE` revisa los videojuegos **fila por fila**, antes de formar los grupos.
- En ese paso todavía no hay un conteo por desarrollador que se pueda comparar con 1.

Ojo con esto

Cambiarlo por `WHERE COUNT(*) > 1` **también da error**: un conteo del grupo no se filtra en `WHERE`.

Esta consulta falla también en SQLite. Aunque permite algunos alias en `WHERE`, no permite allí este alias de `COUNT(*)`.

¿Por qué sí funciona `HAVING COUNT(*) > 1`?

```sql
SELECT developer, COUNT(*) AS total
FROM videogames
GROUP BY developer
HAVING COUNT(*) > 1;
```

Ejemplo: 6 videojuegos en la tabla

Desarrollador	Conteo	¿Se queda?
Aurora	3	Sí: $3 > 1$
Pixel	1	No: $1 \not> 1$
Nube	2	Sí: $2 > 1$

1. `GROUP BY developer` reúne los juegos del mismo desarrollador.
2. `HAVING COUNT(*) > 1` cuenta los juegos de cada grupo y conserva los grupos con más de uno.
3. `SELECT` muestra el desarrollador y su conteo, con el nombre `total`: **Aurora, 3** y **Nube, 2**. El orden no está garantizado sin `ORDER BY`.

La diferencia

`WHERE` decide **qué filas entran al grupo**. `HAVING` decide **qué grupos aparecen en el resultado**.

Modelos de datos

Un mismo diseño, en tres niveles de detalle



- Cada modelo **agrega detalle** al anterior. No se tira nada: se completa.
- Los archivos están en la carpeta `modelado/` de esta sesión.

En palabras simples

Es construir una casa: primero el **dibujo** de qué ambientes hay, luego el **plano** con medidas y, al final, los **planos de obra** con los materiales reales. Ya viviendo en ella, se hacen **mejoras**.

El caso: una peluquería de mascotas

- Se registran **clientes** y sus **mascotas**.
- Una **cita** es de una mascota, para un **servicio**, con un **empleado**, en una fecha y hora. Tiene un estado.
- Cada servicio tiene una **duración** y un **precio**.
- Al completarse la cita se registra su **pago**: monto y método.
- El **encargado** inicia sesión con usuario y contraseña.

Fuera de alcance

Varias sedes, inventario, recordatorios, pagos parciales, varias cuentas o roles.

El primer paso

Los **sustantivos importantes** del problema (en negrita) son candidatos a **entidades**. Lo que se dice de ellos, a **atributos**. Los verbos que los unen, a **relaciones**.

Modelo conceptual: qué hay y cómo se relaciona

Describe el negocio con **entidades**, **atributos** y **relaciones**.

- **Entidades:** cliente, mascota, cita, servicio, empleado, pago y encargado.
- **Atributos:** el cliente tiene nombre y teléfono. La mascota tiene nombre, especie y raza. La cita tiene fecha, hora y estado.
- **Relaciones:** un cliente es dueño de muchas mascotas. Una mascota puede tener muchas citas. Cada cita corresponde a un servicio y un empleado.
- **Regla del negocio:** al completar una cita se registra su pago total, con monto y método.

Qué define este nivel

Explica **qué información necesita la peluquería** y cómo se relaciona. Todavía no elige tipos SQL, claves foráneas ni gestor.

Modelo lógico: tablas, columnas y claves

Organiza la información en **tablas** y define cómo identificar y relacionar sus filas.

- **Tablas y columnas:** `cliente` guarda `id`, `nombre` y `telefono`. `mascota` agrega `cliente_id` a sus datos.
- **Clave primaria (PK):** `id` identifica cada fila de una tabla.
- **Clave foránea (FK):** `mascota.cliente_id` referencia a `cliente.id`. La tabla `cita` referencia a mascota, servicio y empleado.
- **Dominios y reglas:** se distingue texto, números y fechas. Se indica qué datos son obligatorios y cuáles no se pueden repetir.

Qué define este nivel

Establece la **estructura y las restricciones**. En el físico se eligen los tipos y la sintaxis exacta de cada gestor.

Conceptual, lógico y físico: lado a lado

	Qué muestra	La cita, en ese modelo	¿Depende del gestor?
Conceptual	entidades, atributos y relaciones con verbos	«una mascota *tiene* muchas citas»	No
Lógico	tablas y columnas; claves, tipos y reglas	`cita(id PK, fecha_hora, …, mascota_id FK)`	No
Físico	el SQL exacto que crea la base	`CREATE TABLE cita (…);`	**Sí**

La idea clave

El conceptual y el lógico son **los mismos** para SQL Server, MySQL o SQLite. Solo el **físico** cambia según el gestor.

En el curso: conceptual y lógico en **DBML** (dbdiagram), físico en un archivo `.sql`.

Modelo físico

Modelo físico: el SQL que crea la base de datos

```sql
CREATE TABLE cita (
    id INTEGER PRIMARY KEY,
    fecha_hora TEXT NOT NULL,
    estado TEXT NOT NULL DEFAULT 'programada'
        CHECK (estado IN ('programada', 'completada',
                          'cancelada')),
    mascota_id  INTEGER NOT NULL,
    servicio_id INTEGER NOT NULL,
    empleado_id INTEGER NOT NULL,
    FOREIGN KEY (mascota_id) REFERENCES mascota(id),
    -- … y lo mismo para servicio y empleado
    -- un empleado no atiende dos citas a la misma hora
    UNIQUE (empleado_id, fecha_hora)
);
```

- Es el archivo `4_modelo_fisico.sql`.
- Está escrito para **un gestor**: aquí, SQLite.
- Aquí las reglas se vuelven **de verdad**: la base de datos **rechaza** lo que no las cumple.

Cómo se ve en la realidad

Con este archivo, cualquiera crea la misma base de datos **vacía** en su computadora.

Forward Engineering: del modelo a la base de datos

Ingeniería directa

Parte de un **modelo definido para un gestor** y genera el SQL que crea su estructura. Puede exportar un script o ejecutarlo en un servidor.

1. Definir tablas, tipos, claves y restricciones en el modelo.
2. Generar y revisar las instrucciones `CREATE TABLE` y los demás objetos.
3. Ejecutar el script para crear la estructura de la base de datos.

Cómo se ve en la realidad

En MySQL Workbench, el modelo de la peluquería permite generar un script para MySQL. Al ejecutarlo se crean tablas como `cliente`, `mascota` y `cita`.

Generar las tablas **no llena automáticamente los datos** de clientes o citas.

Reverse Engineering: de la base de datos al modelo

Ingeniería inversa

Parte de una **base de datos existente** o de su script de creación y reconstruye un modelo de su estructura.

1. Leer las tablas, columnas, claves y restricciones existentes.
2. Importar esa estructura en una herramienta de modelado.
3. Revisar el modelo obtenido para documentar o modificar la base.

Cómo se ve en la realidad

Si recibimos la base MySQL de la peluquería sin su modelo, Workbench puede importar el esquema y mostrar las relaciones declaradas con claves foráneas.

Ojo con esto

Recupera lo que está definido en la base. Las reglas que solo conoce la aplicación y el significado del negocio requieren revisión humana.

MySQL, SQL Server y SQLite: resumen

MySQL	Gestor relacional que funciona como **servidor**. **InnoDB** es su motor de almacenamiento predeterminado: admite transacciones y claves foráneas. **MySQL Workbench** permite administrar bases y trabajar con modelos.
SQL Server	Gestor relacional de **Microsoft**, también con servidor. Usa **Transact-SQL (T-SQL)**. **SSMS** es una herramienta para conectarse, ejecutar consultas y administrar las bases.
SQLite	Motor integrado en la propia aplicación, **sin un servidor separado**. Guarda la base en un archivo local. **DB Browser for SQLite** permite abrirla y trabajar con sus tablas.

En palabras simples

El gestor ejecuta SQL y administra los datos. Workbench, SSMS y DB Browser son programas que permiten trabajar con esos gestores.

Los tres gestores, lado a lado

	SQL Server	MySQL	SQLite
¿Hay servidor?	sí	sí	**no**
Programa para usarlo	SSMS	MySQL Workbench	DB Browser for SQLite
Su SQL se llama	Transact-SQL	SQL de MySQL	SQL de SQLite
Dónde se guarda	`.mdf` y `.ldf`	carpeta de datos; `.ibd` por tabla*	un archivo, p. ej. `.db`
Numerar solo el `id`	`IDENTITY(1,1)`	`AUTO_INCREMENT`	`INTEGER PRIMARY KEY`
Las 5 primeras filas	`SELECT TOP 5 …`	`… LIMIT 5`	`… LIMIT 5`

*MySQL: InnoDB con un archivo por tabla. SQLite puede crear archivos auxiliares.
Las 5 primeras filas requieren `ORDER BY` para definir un orden predecible.

El modelo conceptual y el lógico de la peluquería **no cambian**. Para cada gestor solo se reescribe el **físico**.

Preguntas



Cierre: lo de hoy en una tabla

	En una frase
Orden de `SELECT`	El orden escrito difiere del orden lógico.
Modelo conceptual	Entidades, atributos y relaciones del negocio.
Modelo lógico	Tablas, columnas, claves y restricciones.
Modelo físico	Tipos y SQL concretos para el gestor elegido.
Forward Engineering	Genera la estructura de la base a partir del modelo.
Reverse Engineering	Reconstruye un modelo a partir de una estructura existente.
Gestores	SQL Server y MySQL usan servidor. SQLite se integra en la aplicación.

El conceptual y el lógico de la peluquería **se mantienen**. Adaptamos el **físico** a cada gestor.

Referencias

- Coronel, C., Morris, S., & Rob, P. (2011). *Bases de datos: diseño, implementación y administración*. Cengage Learning.
- Microsoft Learn. *SELECT (Transact-SQL): orden lógico de procesamiento*.
- Oracle. MySQL Workbench Manual: *Forward Engineering* y *Reverse Engineering*.
- Oracle. MySQL Reference Manual: *File-Per-Table Tablespaces*.
- Microsoft Learn. *Database Files and Filegroups*.
- SQLite. *Appropriate Uses For SQLite* y *Autoincrement*.



UTP Universidad Tecnológica del Perú